

VueJS Router

- Router Concepts
- Example for Simple routing
- Example for Nested routing
- Example Use case for routing

Why Router in VueJS?

To allow a [Vue.js application](#) to have multiple pages/views and navigate [between them](#) without reloading the entire web page.

Suppose your application has:

- Home
- Students
- About

Vue Router – helps to navigate between the views without reloading the entire application

Vue Router maps URL paths to Vue components:

URL	Component

/	→ Home.vue
/students	→ Students.vue
/about	→ About.vue

`npm install vue-router` means:

Download and add Vue Router to your Vue.js project so that the project can use routing.

Vue Router provides:

- **Navigation between views**

- / → Home

- /student → Student

- /about → About

- **URL-based access**

- A user can directly enter: `http://localhost:5173/student` and Vue Router displays `Student.vue`.

- **Single Page Application (SPA) behavior**

- The application is loaded once, and different components are displayed as the URL changes.

- **Browser Back/Forward support**

- The browser's Back and Forward buttons work with the application's routes.

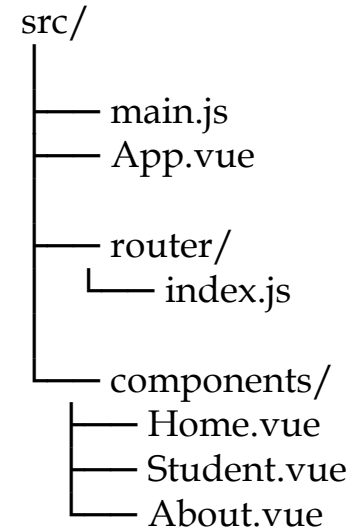
router/index.js

- defines which URL should display which component.

```
import { createRouter, createWebHistory } from 'vue-router'
import Home from '../components/Home.vue'
import Student from '../components/Student.vue'
import About from '../components/About.vue'
const routes = [
  {
    path: '/',
    component: Home
  },
  {
    path: '/students',
    component: Student
  },
  {
    path: '/about',
    component: About
  }
]
const router = createRouter({
  history: createWebHistory(),
  routes
})
export default router
```

Ex1: Router

Project Structure



`createRouter({...})` - Vue Router function that **creates/configures the router** for your application.history:
`createWebHistory()` - tells Vue Router how to **manage the browser URL/history**.

Home.vue

```
<template>  
  <h1>Home  
Page</h1>  
</template>
```

About.vue

```
<template>  
  <h1>About Page</h1>  
</template>
```

Student.vue

```
<template>  
  <h1>Students Page</h1>  
</template>
```

main.js

```
import './assets/main.css'  
import router from './router'  
import { createApp } from 'vue'  
import App from './App.vue'  
createApp(App).use(router).mount('#app')
```

My Student Application

Output → [Home](#) | [Student](#) | [About](#)

About Page

App.vue

```
<template>
  <div>
    <h1>My Student Application</h1>
    <nav>
      <RouterLink to="/">Home</RouterLink> |
      <RouterLink to="/student">Student</RouterLink> |
      <RouterLink to="/about">About</RouterLink>
    </nav>
    <hr>
    <RouterView />
  </div>
</template>
```

router-link → **Vue Router's navigation component**
to → **specifies the URL path to navigate to**

<RouterView /> is a **placeholder or display area** in a Vue application where Vue Router displays the component that matches the current URL.

Ex1: Nested Routing

http://localhost:5173/student

main.js

```
import './assets/main.css'
import { createApp } from 'vue'
import App from './App.vue'
createApp(App).use(router).mount('#app')
```

App.vue

```
<template>
  <div>
    <h1>My Student Application</h1>
    <nav>
      <RouterLink to="/">Home</RouterLink> |
      <RouterLink to="/student">Student</RouterLink> |
      <RouterLink to="/about">About</RouterLink>
    </nav>
    <hr>
    <RouterView />
  </div>
</template>
```

My Student Application

[Home](#) | [Student](#) | [About](#)

Student

[Add Student](#) | [Delete Student](#)

My Student Application

[Home](#) | [Student](#) | [About](#)

Student

[Add Student](#) | [Delete Student](#) | [Update Student](#)

Add Student

```
src/
|
├── main.js
├── App.vue
|
├── router/
│   └── index.js
|
└── components/
    ├── Home.vue
    ├── About.vue
    ├── Student.vue
    ├── Add.vue
    └── Delete.vue
```

About.vue

```
<template>
  <h1>About Page</h1>
</template>
```

Add.vue

```
<template>
  <h1>Add Student</h1>
</template>
```

Home.vue

```
<template>
  <h1>Home Page</h1>
</template>
```

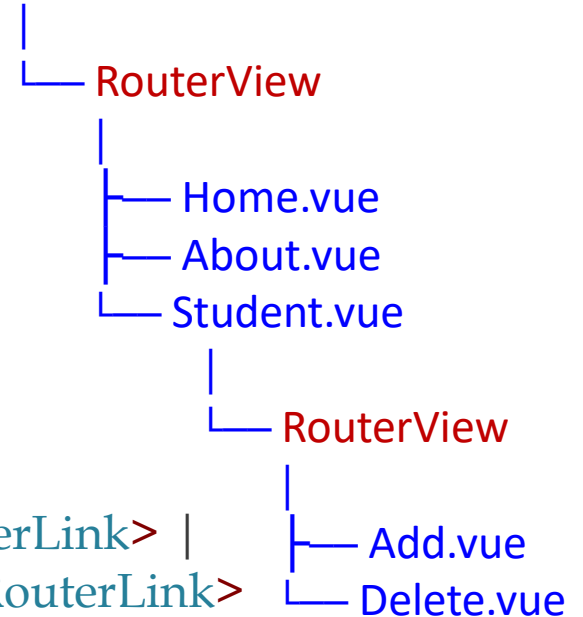
Delete.vue

```
<template>
  <h1>Delete Student</h1>
</template>
```

Student.vue

```
<template>
  <div>
    <h1>Student</h1>
    <nav>
      <RouterLink to="/student/add">Add Student</RouterLink> |
      <RouterLink to="/student/delete">Delete Student</RouterLink>
    </nav>
    <hr>
    <!-- Child components will appear here -->
    <RouterView />
  </div>
</template>
```

App.vue



A college wants to develop a *Student Management Application* using Vue.js. The application should allow the administrator to:

View the list of registered students.

- Add a new student by entering name and email.
- Delete an existing student using the email address.
- Update a student's name and email by providing the existing email address.

The application should use Vue Router with the following routes:

/ → Home

/about → About

/student → Student Management

/student/add → Add Student

/student/delete → Delete Student

/student/update → Update Student

The Add, Delete, and Update components must communicate with the Student component using custom events.

router/index.js

Ex2: Use Case Nested Routing

```
import { createRouter, createWebHistory } from 'vue-router'
import Home from '../components/Home.vue'
import About from '../components/About.vue'
import Student from '../components/Student.vue'
import Add from '../components/Add.vue'
import Delete from '../components/Delete.vue'
import Update from '../components/Update.vue'
const routes = [
  { path: '/', component: Home },
  { path: '/about', component: About },
  { path: '/student', component: Student,
children: [
    { path: 'add', component: Add },
    { path: 'delete', component: Delete },
    { path: 'update', component: Update }
  ]
}
]
const router = createRouter({
  history: createWebHistory(),
  routes
})
export default router
```

App.vue

```
<template>
  <div>
    <h1>My Student Application</h1>
    <nav>
      <RouterLink to="/">Home</RouterLink> |
      <RouterLink to="/student">Student</RouterLink> |
      <RouterLink to="/about">About</RouterLink>
    </nav>
    <hr>
    <RouterView />
  </div>
</template>
```

main.js

```
import './assets/main.css'
import router from './router'

import { createApp } from 'vue'
import App from './App.vue'
createApp(App).use(router).mount('#app')
```

```
<script setup>
import { ref } from 'vue'
const name = ref('')
const email = ref('')
const emit = defineEmits(['add'])
function handleSubmit() {
  emit('add', {
    name: name.value,
    email: email.value,
  })
  name.value = ''
  email.value = ''
}
</script>
<template>
  <h2>Add Student</h2>
  <form @submit.prevent="handleSubmit">
    <input v-model="name" placeholder="Enter your Name" required /><br>
    <input v-model="email" type="email" placeholder="Enter your E-Mail" required /><br>
    <button type="submit">Register</button>
  </form>
</template>
```

```
<script setup>
import { ref } from 'vue'
const email = ref('')
const emit = defineEmits(['delete'])
function handleDelete() {
  emit('delete', email.value)
  email.value = ''
}
</script>
<template>
  <div>
    <h2>Delete Student</h2>
    <form @submit.prevent="handleDelete">
      <input v-model="email" type="email" placeholder="Enter student email"
        required
      />
      <br><br>
      <button type="submit"> Delete </button>
    </form>
  </div>
</template>
```

```

<script setup>
import { ref } from 'vue'
const oldEmail = ref('')
const name = ref('')
const email = ref('')
const emit = defineEmits(['update'])
function handleUpdate() {
  emit('update', {
    oldEmail: oldEmail.value,
    name: name.value,
    email: email.value
  })
  oldEmail.value = ''
  name.value = ''
  email.value = ''
}
</script>

<template>
  <div>
    <h2>Update Student</h2>
    <form @submit.prevent="handleUpdate">
      <input v-model="oldEmail" type="email" placeholder="Enter
existing email"
      required
      />
      <input
        v-model="name" type="text" placeholder="Enter new name"
        required
      />
      <input
        v-model="email" type="email" placeholder="Enter new email"
        required
      />
      <button type="submit"> Update </button>
    </form>
  </div>
</template>

```

```
<script setup>
import { ref } from 'vue'
const students = ref([])
function addStudent(newStudent) {
  students.value.push(newStudent)
}
function deleteStudent(email) {
  students.value = students.value.filter(
    student => student.email !== email
  )
}
function updateStudent(updatedStudent) {
  const student = students.value.find(
    student => student.email === updatedStudent.oldEmail
  )
  if (student) {
    student.name = updatedStudent.name
    student.email = updatedStudent.email
  }
}
</script>
```

students.value.filter(...)

filter() goes through every student in the array and decides whether to keep that student. The condition determines the answer.

```
<template>
  <div>
    <h1>Student</h1>
    <nav>
      <RouterLink to="/student/add">Add Student</RouterLink> |
      <RouterLink to="/student/delete">Delete Student</RouterLink> |
      <RouterLink to="/student/update">Update Student</RouterLink>
    </nav>
    <hr>
    <!-- Child components will appear here -->
    <RouterView @add="addStudent" @delete="deleteStudent"
@update="updateStudent"/>
    <ul>
      <li v-for="student in students" :key="student">{{ student.name
}} - {{ student.email }}</li>
    </ul>
  </div>
</template>
```


My Student Application

[Home](#) | [Student](#) | [About](#)

Student

[Add Student](#) | [Delete Student](#) | [Update Student](#)

Add Student

Register

- bp - bp@gmail.com
- ms - ms@gmail.com
- msb - msb@gmail.com



My Student Application

[Home](#) | [Student](#) | [About](#)

Student

[Add Student](#) | [Delete Student](#) | [Update Student](#)

Add Student

Register

- bp - bp@gmail.com
- ms - ms@gmail.com
- vsf - vsf@gmail.com

My Student Application

[Home](#) | [Student](#) | [About](#)

Student

[Add Student](#) | [Delete Student](#) | [Update Student](#)

Delete Student

Delete

- bp - bp@gmail.com
- ms - ms@gmail.com
- msb - msb@gmail.com



My Student Application

[Home](#) | [Student](#) | [About](#)

Student

[Add Student](#) | [Delete Student](#) | [Update Student](#)

Update Student

Update

- bp - bp@gmail.com
- ms - ms@gmail.com
- vsf - vsf@gmail.com



My Student Application

[Home](#) | [Student](#) | [About](#)

Student

[Add Student](#) | [Delete Student](#) | [Update Student](#)

Delete Student

Delete

- bp - bp@gmail.com
- ms - ms@gmail.com

My Student Application

[Home](#) | [Student](#) | [About](#)

Student

[Add Student](#) | [Delete Student](#) | [Update Student](#)

Update Student

Update

- Prabavathy - Prabavathy@gmail.com
- ms - ms@gmail.com
- vsf - vsf@gmail.com